Python Libraries for Cybersecurity Tasks

Introduction

Python provides many built-in and third-party libraries that support cybersecurity activities. These libraries help security professionals automate repetitive tasks, analyze system logs, monitor network connections, verify data integrity, and detect suspicious activities.

Examples include os, sys, re, json, hashlib, socket, subprocess, logging, and datetime.

Overview of the Libraries

Library	Main Function	Cybersecurity Application
os	Interacting with the operating system	Checking security log files
sys	Accessing Python runtime information	Reading command-line arguments
re	Pattern matching in text	Extracting IP addresses from logs
json	Processing structured data	Analyzing security events
hashlib	Calculating cryptographic hashes	Verifying file integrity
socket	Network communication	Testing TCP connectivity
subprocess	Running external programs	Automating defensive tools

logging	Recording events and errors	Creating security alerts
datetime	Handling dates and times	Analyzing security event timestamps

1. The os Library

Operating System Interface

The os library allows Python programs to interact with the operating system. It provides functions for working with files, directories, paths, and other operating-system resources.

In cybersecurity, os can be used to check whether security log files exist, inspect file sizes, organize digital evidence, and monitor important files.

Important Functions

Function	Description
os.getcwd()	Returns the current working directory
os.listdir()	Lists the contents of a directory
os.path.exists()	Checks whether a path exists
os.path.getsize()	Returns the file size in bytes
os.makedirs()	Creates directories

Example 1: Checking Whether a Security Log Exists

```
import os
file_path = "security.log"
if os.path.exists(file_path):
    print("Security log file exists.")
else:
    print("Security log file not found.")
```

Example 2: Checking the Size of a Security Log

```
import os
file_path = "security.log"
if os.path.exists(file_path):
    size = os.path.getsize(file_path)
    print("File size:", size, "bytes")
else:
    print("File does not exist.")
```

Possible output:

File size: 2048 bytes

The output depends on the actual file size.

2. The sys Library

System-Specific Parameters and Functions

The sys library provides access to information about the Python interpreter and its runtime environment. It allows programs to read command-line arguments, inspect the Python version, and terminate execution when necessary.

In cybersecurity, it can be used to create command-line tools that accept log file names or other input parameters.

Important Functions and Attributes

Function or Attribute	Description
sys.argv	Contains command-line arguments
sys.exit()	Terminates the program
sys.version	Returns Python version information
sys.platform	Identifies the operating-system platform

Example 1: Displaying the Python Version

```
import sys

print("Python version:")

print(sys.version)

print("Operating platform:")

print(sys.platform)
```

Possible output:

```
Python version:

3.12.0 ...
```

```
Operating platform:
win32
```

The actual output depends on the installed Python version and operating system.

Example 2: Passing a Log File Name to a Program

Save the following code as analyze.py:

```
import sys
import os
if len(sys.argv) < 2:
    print("Usage: python analyze.py security.log")
    sys.exit(1)
file_path = sys.argv[1]
if not os.path.isfile(file_path):
    print("Error: Log file not found.")
    sys.exit(1)
print("Analyzing:", file_path)
```

Run the program from the terminal:

```
python analyze.py security.log
```

Cybersecurity application: This technique is useful for developing command-line tools that analyze login records, inspect files, and automate defensive security tasks.

3. The re Library

Regular Expressions

The re library is used to search, extract, match, and replace text according to specified patterns. These patterns are called regular expressions (regex).

In cybersecurity, regular expressions are useful for analyzing system logs, extracting IP addresses, identifying error messages, and detecting failed login attempts.

Important Functions

Function	Description
re.search()	Searches for the first match
re.findall()	Returns all matches
re.match()	Checks for a match at the beginning of a string
re.sub()	Replaces matching text

Example 1: Extracting IP Addresses from a Security Log

```
import re

log = """
Failed login from 192.168.1.10
Successful login from 10.0.0.5
Failed login from 172.16.0.20
"""

pattern = r"\b(?:\d{1,3}\.){3}\d{1,3}\b"
```

```
ip_addresses = re.findall(pattern, log)

print(ip_addresses)
```

Output:

['192.168.1.10', '10.0.0.5', '172.16.0.20']

Important: This pattern identifies text that resembles an IPv4 address, but it does not validate every address correctly. For example, it may match 999.999.999.999, which is not a valid IPv4 address.

Example 2: Detecting Failed Login Attempts

```
import re

log = """

User Ali: LOGIN_FAILED

User Sara: LOGIN_SUCCESS

User Admin: LOGIN_FAILED

"""

failed_logins = re.findall(r"LOGIN_FAILED", log)

print("Failed login attempts:", len(failed_logins))
```

Output:

Failed login attempts: 2

Explanation: The program searches for every occurrence of LOGIN_FAILED and counts the matches.

Cybersecurity application: Regular expressions can be used to analyze real log files and identify repeated failed login attempts that may indicate a brute-force attack.

4. The json Library

JavaScript Object Notation

The json library allows Python programs to read and write data in JSON format. It converts JSON text into Python objects and converts Python objects into JSON text.

JSON is widely used by security applications, web APIs, Security Information and Event Management (SIEM) systems, and security event logs.

Important Functions

Function	Description
json.loads()	Converts JSON text into a Python object
json.dumps()	Converts a Python object into JSON text
json.load()	Reads JSON data from a file
json.dump()	Writes JSON data to a file

Example 1: Processing a Security Event in JSON Format

```
import json

event = """
{
   "user": "Ali",
   "ip": "192.168.1.10",
   "failed_attempts": 6,
```

```
    "status": "LOGIN_FAILED"
}
"""
data = json.loads(event)
print("User:", data["user"])
print("IP address:", data["ip"])
print("Failed attempts:", data["failed_attempts"])
```

Output:

```
User: Ali
IP address: 192.168.1.10
Failed attempts: 6
```

After conversion, individual values can be accessed through their keys.

Example 2: Generating an Alert When Failed Attempts Exceed a Threshold

```
import json
event = '{"user": "Admin", "failed_attempts": 8}'
data = json.loads(event)
if data["failed_attempts"] >= 5:
    print("SECURITY ALERT!")
    print("Possible brute-force attack.")
```

Output:

```
SECURITY ALERT!
```

Possible brute-force attack.

This example assumes that five or more failed attempts should trigger an alert. In a real environment, the threshold should depend on organizational security policies and the surrounding context.

Cybersecurity application: Security events can be received in JSON format, analyzed, and forwarded to a monitoring dashboard or a SIEM system.

5. The hashlib Library

Cryptographic Hashing

The hashlib library calculates cryptographic hash values, also called digital fingerprints, for data. It can help verify whether a file has changed, compare files, and support data-integrity checks.

One commonly used algorithm available through this library is SHA-256.

A hash is not the same as encryption. Encryption is designed to be reversible with the appropriate key, whereas a cryptographic hash is not designed to recover the original data.

Important Functions and Methods

Function or Method	Description
hashlib.sha256()	Creates a SHA-256 hash object
.update()	Adds data to the hash calculation
.hexdigest()	Returns the hash as a hexadecimal string
.digest()	Returns the hash as bytes

Example 1: Calculating the SHA-256 Hash of a String

```
import hashlib
```

```
data = "Cybersecurity"
hash_value = hashlib.sha256(data.encode("utf-8")).hexdigest()
print("SHA-256:", hash_value)
```

SHA-256 produces a 256-bit hash, typically represented by 64 hexadecimal characters.

Example 2: Verifying File Integrity

Suppose we have a file named report.txt and want to calculate its SHA-256 hash.

```
import hashlib
def calculate_sha256(file_path):
    sha256 = hashlib.sha256()
    with open(file_path, "rb") as file:
        for chunk in iter(
            lambda: file.read(4096), b""
        ):
            sha256.update(chunk)
    return sha256.hexdigest()
print(calculate_sha256("report.txt"))
```

Reading the file in chunks avoids loading the entire file into memory, which is useful for large files.

How Can We Detect File Changes?

1. Calculate the file's hash when the original file is trusted.
2. Store the reference hash in a protected location.

3. Calculate the hash again later.
4. Compare the two hash values.

If the hashes differ, the file contents no longer match the reference version.

However, if an attacker can modify both the file and its stored reference hash, this comparison alone may not be sufficient. Protecting the reference value or using a digital signature can provide stronger assurance.

Cybersecurity application: Verifying the integrity of security logs, backup files, executable files, and digital evidence during digital forensic investigations.

6. The socket Library

Network Communication

The socket library enables Python programs to communicate over networks using protocols such as TCP and UDP.

In cybersecurity, sockets can be used to test whether a network service is reachable, understand client-server communication, and develop simple network monitoring tools.

Important Functions and Methods

Function or Method	Description
socket.socket()	Creates a socket
connect()	Connects to a server
sendall()	Sends data

recv()	Receives data
close()	Closes a connection
settimeout()	Sets a connection timeout

Example 1: Testing TCP Connectivity to a Port

```
import socket
host = "example.com"
port = 443
try:
    with socket.create_connection(
        (host, port), timeout=5
    ):
        print("Connection successful.")
except OSError:
    print("Connection failed or timed out.")
```

Possible output:

Connection successful.

This output appears if the server is reachable and the connection is permitted.

Important: A successful TCP connection does not prove that the HTTPS service is functioning correctly. It only confirms that the TCP connection could be established.

Example 2: Resolving a Domain Name to an IP Address

```
import socket
domain = "example.com"
try:
    ip_address = socket.gethostbyname(domain)
    print("Domain:", domain)
    print("IP address:", ip_address)
except socket.gaierror:
    print("DNS resolution failed.")
```

The returned IP address may vary depending on DNS responses and network configuration.

The function socket.gethostbyname() uses the Domain Name System (DNS) to resolve a hostname to an IPv4 address.

Cybersecurity application: Testing connectivity to authorized services, troubleshooting network problems, and studying network communications. Only test systems that you own or have permission to assess.

7. The subprocess Library

Subprocess Management

The subprocess library allows Python programs to execute external programs and operating-system commands and retrieve their results.

In cybersecurity, it can support defensive automation, system information collection, and integration with other security tools.

Important Functions and Attributes

Function or Attribute	Description

subprocess.run()	Executes a command and waits for it to finish
capture_output=True	Captures standard output and standard error
text=True	Returns captured output as text
returncode	Provides the process exit code
check=True	Raises an exception if the command fails

Example 1: Checking the Python Version Using subprocess

```
import subprocess
import sys
result = subprocess.run(
    [sys.executable, "--version"],
    capture_output=True,
    text=True,
    check=True
)
print(result.stdout or result.stderr)
```

Possible output:

```
Python 3.12.0
```

Example 2: Running a System Command Safely

On Windows, the ipconfig command displays network configuration information.

```
import subprocess
try:
    result = subprocess.run(
        ["ipconfig"],
        capture_output=True,
        text=True,
        timeout=10,
        check=True
    )
    print(result.stdout)
except subprocess.SubprocessError as error:
    print("Command failed:", error)
```

8. The logging Library

Application and Security Logging

The logging library records program messages, events, warnings, and errors. It helps developers understand what happened, when it happened, and why an application failed.

In cybersecurity, logging is essential for monitoring suspicious activities, recording security alerts, and supporting incident investigations.

Application logging should be distinguished from a Security Information and Event Management (SIEM) system, which can collect and correlate logs from multiple sources.

Logging Levels

Level	Meaning	Example
DEBUG	Detailed diagnostic information	Values used during testing
INFO	Normal operational information	Starting a file scan
WARNING	An unexpected condition requiring attention	Repeated failed login attempts
ERROR	A task has failed	Unable to read a file
CRITICAL	A serious problem affecting operation	Failure of a critical component

Example 1: Logging Failed Login Attempts

```
import logging
logging.basicConfig(
    filename="security.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
user = "Admin"
failed_attempts = 6
if failed_attempts >= 5:
    logging.warning(
```

```
        "Multiple failed login attempts detected."
    )
print("Security monitoring completed.")
```

The program creates a file named security.log in the current working directory, provided the file can be created and written to.

Example log entry:

2026-10-17 12:00:00,000 - WARNING - Multiple failed login attempts detected.

The timestamp above is illustrative; the program records the actual execution time.

Example 2: Logging File-Reading Errors

```
import logging
logging.basicConfig(
    filename="errors.log",
    level=logging.ERROR
)
try:
    with open(
        "missing.log", "r", encoding="utf-8"
    ) as file:
        content = file.read()
except OSError:
    logging.exception("Failed to read the log file.")
```

```
print("An error occurred. Check errors.log.")
```

If the file does not exist or cannot be read, the program records the error and exception details in errors.log.

Cybersecurity application: Recording security alerts, monitoring failures, and defensive scanning results. Log files should be protected against unauthorized modification, access permissions should be properly configured, and sensitive information should be excluded.

9. The datetime Library

Date and Time Management

The datetime library provides tools for handling dates, times, timestamps, and time intervals.

In cybersecurity, timestamps help analysts reconstruct the sequence of events, calculate the intervals between login attempts, and correlate logs from different systems.

Important Functions and Classes

Function or Class	Description
datetime.now()	Returns the current local date and time
datetime.now(timezone.utc)	Returns the current time in UTC
datetime.strptime()	Converts a string into a date-time object
strftime()	Converts a date-time object into a formatted string

timedelta	Represents a duration or time interval

Example 1: Recording the Time of a Security Event

```
from datetime import datetime, timezone
event_time = datetime.now(timezone.utc)
print("Security event time:", event_time)
```

Possible output:

```
Security event time: 2026-10-17 09:20:00+00:00
```

Third-Party Functions in Python

1. Introduction

Third-party functions are functions provided by external libraries that are not included in Python's standard library. They allow programmers to perform specialized tasks without writing all the code from scratch.

Third-party functions are widely used in cybersecurity, data analysis, artificial intelligence, web development, and network programming.

Examples of Third-Party Libraries and Functions

Library	Purpose	Example Function
Requests	Sending HTTP requests to websites	requests.get(url)
NumPy	Mathematical operations and array processing	numpy.mean(data)
Pandas	Data analysis and processing	pandas.read_csv("data.csv")
Scikit-learn	Machine learning and predictive analysis	model.predict(data)
Cryptography	Encrypting and decrypting data	Encryption and decryption functions

Installing Third-Party Libraries

Before using a third-party library, we may need to install it using the pip package manager.

For example, to install the requests library, run the following command in the terminal or Command Prompt:

pip install requests

After installation, the library can be imported into a Python program.

1. Requests Library

The requests library allows Python programs to communicate with websites and web services through HTTP requests.

Example

```
import requests

response= requests.get("https://example.com")

print(response.status_code)
```

Expected output:

200

2. NumPy Library

NumPy is used for numerical calculations and working with arrays.

```
import numpy as np

numbers = [10, 20, 30, 40, 50]

average = np.mean(numbers)

print("Average:", average)
```

Output:

Average: 30.0

Explanation:

- import numpy as np imports NumPy using the alias np.
- np.mean(numbers) calculates the average of the numbers.

- print() displays the result.

3. Pandas Library

Pandas is commonly used to organize, process, and analyze structured data.

```
import pandas as pd
data = {
    "User": ["Ali", "Sara", "Omar"],
    "Failed_Logins": [1, 2, 7]
}
df = pd.DataFrame(data)
print(df)
```

Output:

```
  User  Failed_Logins
0   Ali             1
1  Sara             2
2  Omar             7
```

Explanation:

- pd.DataFrame() creates a table from the supplied data.
- Each row represents a user.
- The Failed_Logins column contains the number of failed login attempts.

In cybersecurity, Pandas can help analyze authentication logs and identify unusual activity.

4. Scikit-learn Library

Scikit-learn provides tools for machine learning, including classification, clustering, and prediction.

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()

print("Machine learning model created")
```

Explanation:

- DecisionTreeClassifier is a machine learning algorithm used for classification.
- model = DecisionTreeClassifier() creates a classifier object.
- The model must be trained with suitable data before it can make meaningful predictions.

This library can be used to develop systems that classify network traffic or detect suspicious activities.

5. Cryptography Library

The cryptography library in Python provides tools for protecting sensitive information using encryption, decryption, digital signatures, and other cryptographic operations.

It is a third-party library, so it usually needs to be installed before use.

Installing the Cryptography Library

Run the following command in the terminal or Command Prompt:

```
pip install cryptography
```

Important Cryptography Functions

Function / Class	Purpose
Fernet.generate_key()	Generates a secret encryption key
Fernet()	Creates an object for encryption and decryption
encrypt()	Encrypts data
decrypt()	Decrypts previously encrypted data
hashes.Hash()	Computes a cryptographic hash
AESGCM()	Encrypts and decrypts data using AES-GCM
sign()	Creates a digital signature using supported private-key objects
verify()	Verifies a digital signature

Note: Some entries are classes or methods rather than standalone functions.

Example: Encryption and Decryption

This example uses Fernet to encrypt and decrypt a message.

```
from cryptography.fernet import Fernet

# Generate an encryption key

key = Fernet.generate_key()

# Create a Fernet object

cipher = Fernet(key)

# Original message

message = b"Confidential Data"

# Encrypt the message
```

```
encrypted = cipher.encrypt(message)
print("Encrypted:", encrypted)
# Decrypt the message
decrypted = cipher.decrypt(encrypted)
print("Decrypted:", decrypted.decode())
```

Explanation

1. Fernet.generate_key() generates a secret key.
2. Fernet(key) creates an encryption and decryption object.
3. cipher.encrypt(message) encrypts the original message.
4. cipher.decrypt(encrypted) decrypts the encrypted data.
5. .decode() converts the resulting bytes into readable text.

Expected output:

Advantages of Third-Party Libraries

The main advantages include:

1. **Time Saving:** Reduce the amount of code developers need to write.
2. **Code Reusability:** Provide functions and classes that can be reused in different projects.
3. **Advanced Functionality:** Support specialized tasks such as machine learning and cryptography.
4. **Productivity:** Make it easier to develop complex applications.
5. **Community Support:** Many popular libraries have documentation, tutorials, and active developer communities.

Limitations and Security Considerations

Although third-party libraries are useful, developers should consider the following issues:

- **Dependencies:** A library may require additional packages.
- **Compatibility:** Some libraries may not support every Python version.
- **Security Vulnerabilities:** Outdated or vulnerable packages may introduce security risks.
- **Trustworthiness:** Libraries should be obtained from reliable sources.
- **Maintenance:** Developers should check whether a library is actively maintained.

Use a virtual environment to isolate project dependencies, and keep packages updated when appropriate.

Third-Party Functions in Cybersecurity

Third-party libraries provide useful functions for automating cybersecurity tasks.

Examples include:

- Network Security: Using requests to communicate with web services.
- Data Analysis: Using Pandas to analyze security logs.
- Machine Learning: Using Scikit-learn to detect suspicious patterns.
- Cryptography: Using the cryptography library to protect sensitive information.
- Network Programming: Using libraries such as Scapy to inspect and analyze network packets.